SIMATS ENGINEERING

CSA09 – PROGRAMMING IN JAVA

ASSIGNMENT 4 – CO4

Event-Driven GUI Design – AWT, Swing and GUI–JDBC Integration

Name	Sanjay Adithya S J
Register Number	192311336
Course Code	CSA0921
Course Outcome	CO4 – Event delegation model, listeners, layouts, AWT/Swing, GUI–JDBC
GitHub Repository	https://github.com/sanoryx/CSA0921-JAVA

AIM AND THEORY

Aim: To design event-driven user interfaces before building them, to keep the interface separate from application logic, and to implement, trace and test them with AWT and Swing (including a Swing–JDBC application). **Theory:** In the **event delegation model** an *event source* (Button, TextField, Frame…) generates an *event object* (ActionEvent, MouseEvent, WindowEvent…) and delivers it to registered *listener* objects that implement a listener interface (ActionListener, MouseListener…). Adapter classes (MouseAdapter, WindowAdapter) provide empty implementations so that only the needed methods are overridden. Listeners may be a class implementing the interface, a named inner class, an anonymous inner class or a lambda. Layout managers (FlowLayout, BorderLayout, GridLayout, CardLayout, GridBagLayout) compute component sizes and positions. All Swing components must be created and changed on the **event-dispatch thread** (EDT); long work belongs in a *SwingWorker*. **Environment note:** the programs were run on OpenJDK 21 with a virtual X display (Xvfb); the pictures are real captures of the running windows (no window manager, therefore no title bars). The MySQL database server was not available in the sandbox, so the GUI–JDBC application was tested against SQLite through the same JDBC code; the MySQL script is supplied.

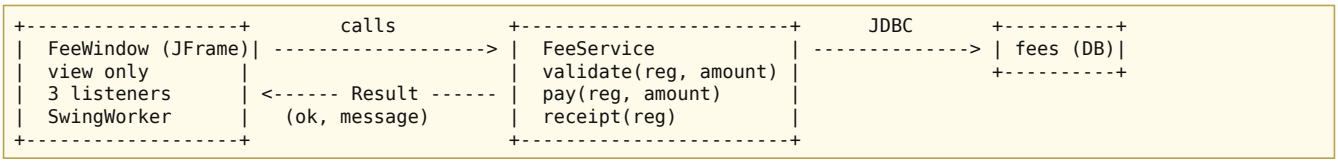
TASK 1 – GUI Design Review: Diagnose and Repair a Poor Event-Handling Design (6 marks)

Task 1.1 – Diagnose the design (2)

#	Deficiency	What the user experiences / what can go wrong
1	Interface and logic mixed in one actionPerformed()	Impossible to unit-test without opening a window; any change to payment rules risks breaking the UI.
2	`e.getActionCommand() == "Pay"` compares strings with ==	Compares references; works only by luck of string interning, fails when the label changes or is built at run time → the button silently does nothing or always runs the else branch (Clear).
3	No validation of input: Double.parseDouble(amount.getText())	Empty or non-numeric text throws NumberFormatException (uncaught): the click appears to do nothing; negative amounts and blank register numbers are accepted.
4	Database work and Thread.sleep(3000) on the event-dispatch thread	The window freezes for 3 s plus connection time; it cannot repaint, move or close; the user clicks again → a double payment.
5	No feedback to the user (System.out.println("Paid"))	The user does not know whether the payment worked; the message goes to the console, not the window.
6	Receipt button has no handler and falls into the else branch	Pressing Receipt clears the fields and the data the user typed is lost.
7	SQL built by concatenation	SQL injection: a register number such as `x'); DROP TABLE fees;--` alters the statement; also fails on names with a quote.
8	Connection and Statement never closed; window close not handled	Connection leak (the database runs out of connections); clicking the close box does nothing (no WindowListener).
9	Checked exceptions (SQLException, InterruptedException) not handled – the code does not compile as written	The programmer is forced to add throws/try; a thrown exception is lost or kills the EDT with a stack trace the user never sees.
10	One listener for all buttons with an if–else chain; credentials (URL, USER, PWD) in the UI class	Chain grows with each button; the UI knows database secrets.

Task 1.2 – Repair the design (3)

Class structure: the window only collects input and shows results; FeeService validates and pays



Source file: FeeService.java

```
import java.sql.*;

/** Business logic: validation and payment. No AWT/Swing imports - can be unit-tested without a window. */
public class FeeService {
    public record Result(boolean ok, String message) {}
    private final String url, user, pwd;
    public FeeService(String url, String user, String pwd) { this.url = url; this.user = user; this.pwd = pwd; }

    public Result validate(String regNo, String amountText) {
        if (regNo == null || !regNo.matches("\\d{9}")) return new Result(false, "Register number must be 9 digits");
        if (amountText == null || amountText.isBlank()) return new Result(false, "Enter the amount");
        try {
            double a = Double.parseDouble(amountText.trim());
            if (a <= 0 || a > 500000) return new Result(false, "Amount must be between 1 and 500000");
        } catch (NumberFormatException e) { return new Result(false, "Amount must be a number"); }
        return new Result(true, "");
    }
    /** Blocking (database) call - the window calls this from a SwingWorker, never from the event thread. */
    public Result pay(String regNo, String amountText) {
        Result v = validate(regNo, amountText);
        if (!v.ok()) return v;
        double amount = Double.parseDouble(amountText.trim());
        String sql = "INSERT INTO fees(reg_no, amount) VALUES (?, ?)";               // PreparedStatement: no SQL injection
        try (Connection c = (user == null ? DriverManager.getConnection(url) : DriverManager.getConnection(url, user, pwd));
             PreparedStatement ps = c.prepareStatement(sql)) {                       // try-with-resources closes the
                 connection
            ps.setString(1, regNo); ps.setDouble(2, amount);
            ps.executeUpdate();
            return new Result(true, "Paid Rs " + amount + " for " + regNo);
        } catch (SQLException e) { return new Result(false, "Payment failed: " + e.getMessage()); }
    }
    public Result receipt(String regNo) {
        try (Connection c = (user == null ? DriverManager.getConnection(url) : DriverManager.getConnection(url, user, pwd));
             PreparedStatement ps = c.prepareStatement("SELECT SUM(amount) FROM fees WHERE reg_no = ?")) {
            ps.setString(1, regNo); ResultSet rs = ps.executeQuery(); rs.next();
            double total = rs.getDouble(1);
            return total == 0 ? new Result(false, "No payments found for " + regNo) : new Result(true, "Receipt: total paid
                Rs " + total + " by " + regNo);
        } catch (SQLException e) { return new Result(false, "Receipt failed: " + e.getMessage()); }
    }
}
```

Source file: FeeWindow.java

```
import javax.swing.*;
import java.awt.*;

/** View only: collects input and shows results. Each button has its own listener. */
public class FeeWindow extends JFrame {
    private final JTextField regNo = new JTextField(12), amount = new JTextField(12);
    private final JLabel status = new JLabel("Ready");
    private final JButton pay = new JButton("Pay"), clear = new JButton("Clear"), receipt = new JButton("Receipt");
    private final FeeService service;

    public FeeWindow(FeeService service) {
        super("Fee Payment");
        this.service = service;
        setLayout(new GridLayout(4, 2, 6, 6));
        add(new JLabel("Register No")); add(regNo); add(new JLabel("Amount (Rs)")); add(amount);
        add(pay); add(clear); add(receipt); add(status);
        pay.addActionListener(e -> doPay());
        clear.addActionListener(e -> { regNo.setText(""); amount.setText(""); status.setText("Cleared"); });
        receipt.addActionListener(e -> run(() -> service.receipt(regNo.getText().trim())));
        setDefaultCloseOperation(DISPOSE_ON_CLOSE);
        pack();
        setSize(400, 170);
    }
    private void doPay() {
        FeeService.Result v = service.validate(regNo.getText().trim(), amount.getText());   // fast, safe on the event thread
        if (!v.ok()) { status.setText(v.message()); return; }
        run(() -> service.pay(regNo.getText().trim(), amount.getText()));
    }
    /** Runs the slow work in a background thread; updates the label back on the event thread. */
    private void run(java.util.function.Supplier<FeeService.Result> job) {
        pay.setEnabled(false); receipt.setEnabled(false); status.setText("Processing...");
        new SwingWorker<FeeService.Result, Void>() {
            @Override protected FeeService.Result doInBackground() { return job.get(); }
            @Override protected void done() {
                try { status.setText(get().message()); } catch (Exception ex) { status.setText("Unexpected error"); }
                pay.setEnabled(true); receipt.setEnabled(true);
            }
        }.execute();
    }
    String statusText() { return status.getText(); }

    public static void main(String[] args) throws Exception {
        String url = args.length > 0 ? args[0] : "jdbc:sqlite:fees.db";
        try (var c = java.sql.DriverManager.getConnection(url); var st = c.createStatement()) { st.execute("CREATE TABLE IF
            NOT EXISTS fees(reg_no TEXT, amount REAL)"); }
        SwingUtilities.invokeLater(() -> new FeeWindow(new FeeService(url, null, null)).setVisible(true));
    }
}
```

Test driver used for the screenshots

```
import javax.swing.*;
import java.awt.*;

public class Task1Demo {
    public static void main(String[] args) throws Exception {
        String url = "jdbc:sqlite:" + java.nio.file.Files.createTempFile("fees", ".db");
        try (var c = java.sql.DriverManager.getConnection(url); var st = c.createStatement()) { st.execute("CREATE TABLE
            fees(reg_no TEXT, amount REAL)"); }
        FeeService svc = new FeeService(url, null, null);
        System.out.println("== FeeService unit tests (no window needed) ==");
        String[][] t = {{"192311001", "4500"}, {"19231", "4500"}, {"192311001", ""}, {"192311001", "abc"}, {"192311001",
            "-5"}, {"192311001'; DROP TABLE fees;--", "10"}};
        for (String[] x : t) System.out.printf("  validate(%-34s, %-6s) -> %s%n", "\"" + x[0] + "\"", "\"" + x[1] + "\"",
            svc.validate(x[0], x[1]));
        System.out.println("  pay(192311001, 4500)  -> " + svc.pay("192311001", "4500"));
        System.out.println("  receipt(192311001)    -> " + svc.receipt("192311001"));
        System.out.println("  receipt(192311999)    -> " + svc.receipt("192311999"));
        final FeeWindow[] w = new FeeWindow[1];
        SwingUtilities.invokeAndWait(() -> { w[0] = new FeeWindow(svc); w[0].setLocation(50, 50); w[0].setVisible(true); });
        Thread.sleep(500);
        // drive the window like a user: invalid amount, then valid payment
        JTextField reg = find(w[0], 0), amt = find(w[0], 1);
        SwingUtilities.invokeAndWait(() -> { reg.setText("192311001"); amt.setText("abc"); click(w[0], "Pay"); });
        Thread.sleep(300); Shot.screen(new Rectangle(w[0].getX(), w[0].getY(), w[0].getWidth() + 8, w[0].getHeight() + 30),
            "shots/task1_invalid.png");
        System.out.println("window status after invalid amount: " + w[0].statusText());
        SwingUtilities.invokeAndWait(() -> { amt.setText("4500"); click(w[0], "Pay"); });
        Thread.sleep(800); Shot.screen(new Rectangle(w[0].getX(), w[0].getY(), w[0].getWidth() + 8, w[0].getHeight() + 30),
            "shots/task1_paid.png");
        System.out.println("window status after valid payment : " + w[0].statusText());
        System.exit(0);
    }
    static JTextField find(Container c, int idx) { int n = 0; for (Component k : c.getComponents()) { if (k instanceof
        JRootPane rp) return find(rp.getContentPane(), idx); if (k instanceof JTextField f) { if (n++ == idx) return f; } }
        return null; }
    static void click(Container c, String text) { for (Component k : ((JRootPane)
        c.getComponent(0)).getContentPane().getComponents()) if (k instanceof JButton b && b.getText().equals(text))
        b.doClick(); }
}
```

Execution and output – service tests without a window, then the window driven like a user:

```
$ javac FeeService.java FeeWindow.java Task1Demo.java && java Task1Demo   (virtual display)
== FeeService unit tests (no window needed) ==
  validate("192311001"                      , "4500") -> Result[ok=true, message=]
  validate("19231"                          , "4500") -> Result[ok=false, message=Register number must be 9 digits]
  validate("192311001"                      , ""    ) -> Result[ok=false, message=Enter the amount]
  validate("192311001"                      , "abc" ) -> Result[ok=false, message=Amount must be a number]
  validate("192311001"                      , "-5"  ) -> Result[ok=false, message=Amount must be between 1 and 500000]
  validate("192311001'; DROP TABLE fees;--"  , "10"  ) -> Result[ok=false, message=Register number must be 9 digits]
  pay(192311001, 4500)  -> Result[ok=true, message=Paid Rs 4500.0 for 192311001]
  receipt(192311001)    -> Result[ok=true, message=Receipt: total paid Rs 4500.0 by 192311001]
  receipt(192311999)    -> Result[ok=false, message=No payments found for 192311999]
window status after invalid amount: Amount must be a number
window status after valid payment : Paid Rs 4500.0 for 192311001
```

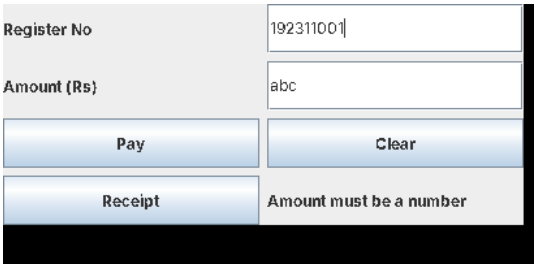


Fig 1.1 – Invalid amount "abc": validated on the EDT, message in the window

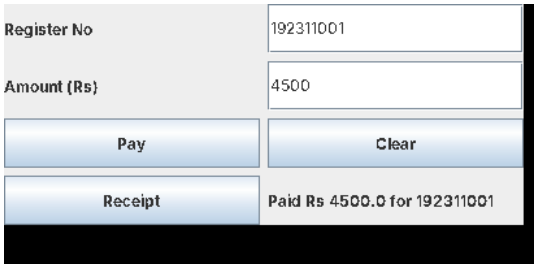


Fig 1.2 – Valid payment: SwingWorker ran the JDBC insert off the EDT

Rewritten event handling: each button has its own lambda listener; Pay validates first (fast, safe on the EDT), disables the buttons, runs *service.pay* in a **SwingWorker** (doInBackground = database, done = update label on the EDT). Receipt now has a real action; Clear only clears.

Task 1.3 – Design defence (1)

The redesign is easier to test because FeeService has no AWT/Swing dependency: validation and payment are tested with plain method calls and a test database (see the test output above), and the window is tested separately. It is easier to maintain because each class has one responsibility, SQL is parameterised, the UI never blocks, and a new payment rule or a new screen (web, mobile) reuses FeeService unchanged. (about 70 words)

Alternative concept: one listener per button (or a Swing Action) instead of an if-else chain

Each button owns a listener object, so dispatch is a direct call: **O(1)**; the if–else chain needs up to **k comparisons** (average k/2) to reach the right branch. The difference is nanoseconds for k = 3, but the chain also forces one class to know every button and be edited for each new one. **Space:** k listener objects (about 16–24 bytes each for lambdas/anonymous classes, plus captured variables) = O(k) versus one listener object for the chain – the extra memory is negligible; a Swing *Action* additionally lets the button, a menu item and a key binding share one object.

TASK 2 – From Requirement to Event-Driven Design: Hospital Outpatient Appointment Desk (8 marks)

Task 2.1 – Event table (2)

Event source	Event object	Listener interface (method)	Action taken
Frame (window)	WindowEvent	WindowListener (windowOpened, windowClosing, windowClosed…)	windowClosing: if a booking is in progress show a confirmation dialog, otherwise dispose()
Patient-ID TextField	TextEvent / ActionEvent (Enter)	TextListener / ActionListener	Validate the ID format; enable the Book button
Department Choice/JComboBox	ItemEvent	ItemListener (itemStateChanged)	Refresh the doctor list for the selected department
Doctor Choice/JComboBox	ItemEvent	ItemListener	Load the available time slots of that doctor
Time-slot List/JList	ItemEvent / ListSelectionEvent	ItemListener / ListSelectionListener	Remember the selected slot
Book Button	ActionEvent	ActionListener (actionPerformed)	Validate, book, show confirmation or error
Cancel Button	ActionEvent	ActionListener	Reset the form
Mouse click on a slot canvas (optional)	MouseEvent	MouseListener / MouseAdapter (mouseClicked)	Select the slot

Task 2.2 – Component hierarchy (2)

Containment tree with the AWT / Swing class used for each node

```
Frame / JFrame  "Outpatient Appointment Desk"           (BorderLayout)
 +-- Panel / JPanel  form (NORTH)                              (GridLayout 5 x 2)
 |     +-- Label / JLabel "Patient ID"        TextField / JTextField
 |     +-- Label / JLabel "Department"        Choice / JComboBox
 |     +-- Label / JLabel "Doctor"            Choice / JComboBox
 |     +-- Label / JLabel "Time slot"         List / JList (in ScrollPane / JScrollPane)
 |     +-- Button "Book"  Button "Cancel"
 +-- TextArea / JTextArea  confirmation or error (CENTER)
 +-- Label / JLabel  status line (SOUTH)
 (dialogs: Dialog / JOptionPane for "Booking in progress - close anyway?")
```

Task 2.3 – Handler logic (3)

Pseudocode – (a) refresh doctor list when the department changes

```
ON ItemEvent FROM departmentChoice            // itemStateChanged(e)
BEGIN
  IF e.stateChange <> SELECTED THEN RETURN         // ignore the DESELECTED event
  dept <- departmentChoice.selectedItem
  doctorChoice.removeAll()
  doctors <- doctorMap.get(dept)                    // Map<String, List<Doctor>> loaded once
  IF doctors = null OR doctors is empty THEN
     doctorChoice.add("No doctor available"); bookButton.disable(); RETURN
  FOR EACH doc IN doctors DO doctorChoice.add(doc.name) END FOR
  slotList.clear(); bookButton.disable()            // a new department invalidates the old slot
END
```

Pseudocode – (b) validate and book the appointment

```
ON ActionEvent FROM bookButton                  // actionPerformed(e)
BEGIN
  id <- patientIdField.text.trim()
  IF id is empty OR id does not match [0-9]{8} THEN show "Enter a valid patient ID"; RETURN
  IF department not selected OR doctor not selected OR slot not selected THEN show "Select department, doctor and
      slot"; RETURN
  bookingInProgress <- TRUE; bookButton.disable(); status <- "Booking..."
  RUN IN BACKGROUND:
      result <- appointmentService.book(id, doctor, slot)      // database work, NOT on the event thread
  WHEN DONE (back on the event thread):
      IF result.ok THEN confirmationArea.text <- "Booked: " + details
      ELSE confirmationArea.text <- "Error: " + result.message   // e.g. slot already taken
      bookingInProgress <- FALSE; bookButton.enable()
END
```

Task 2.4 – Trace of the event sequence (1)

Source file: Task24Trace.java

```
import java.awt.*;
import java.awt.event.*;

/** Task 2.4: logs the order of window-event and paint methods for an AWT Frame, from opening to closing. */
public class Task24Trace extends Frame implements WindowListener {
    static void log(String s) { System.out.println(String.format("%2d. %s", ++n, s)); }
    static int n = 0;
    Task24Trace() {
        super("Appointment Desk");
        log("constructor Task24Trace()");
        addWindowListener(this);
        add(new Label("Appointment desk"));
        setSize(300, 120);
        log("setVisible(true) called");
        setVisible(true);
    }
    @Override public void addNotify() { log("addNotify() - native peer created"); super.addNotify(); }
    @Override public void paint(Graphics g) { log("paint(Graphics)"); super.paint(g); }
    public void windowOpened(WindowEvent e) { log("windowOpened"); }
    public void windowActivated(WindowEvent e) { log("windowActivated"); }
    public void windowDeactivated(WindowEvent e) { log("windowDeactivated"); }
    public void windowIconified(WindowEvent e) { log("windowIconified"); }
    public void windowDeiconified(WindowEvent e) { log("windowDeiconified"); }
    public void windowClosing(WindowEvent e) { log("windowClosing  -> (confirm if a booking is in progress) then dispose()");
         dispose(); }
    public void windowClosed(WindowEvent e) { log("windowClosed"); }
    public static void main(String[] a) throws Exception {
        Task24Trace f = new Task24Trace();
        Thread.sleep(600);
        log("user clicks the close box (WINDOW_CLOSING posted)");
        f.dispatchEvent(new WindowEvent(f, WindowEvent.WINDOW_CLOSING));
        Thread.sleep(400);
        System.exit(0);
    }
}
```

Execution and output – recorded trace of an AWT Frame (opened, painted, closed by the user):

```
$ javac Task24Trace.java && java Task24Trace
 1. constructor Task24Trace()
 2. setVisible(true) called
 3. addNotify() - native peer created
 4. windowOpened
 5. windowActivated
 6. paint(Graphics)
 7. user clicks the close box (WINDOW_CLOSING posted)
 8. windowClosing  -> (confirm if a booking is in progress) then dispose()
 9. windowClosed
```

Order of calls: constructor → setVisible(true) → addNotify (native peer created) → windowOpened → windowActivated → paint → (user action) → windowClosing → dispose() → windowClosed. If the window is minimised the sequence windowIconified/windowDeactivated and later windowDeiconified/windowActivated + repaint is delivered; closing does nothing unless windowClosing calls dispose().

Applet life cycle	Frame/window life cycle
init() – once, when loaded (create components)	constructor / addNotify – create components
start() – when the page is shown or the applet resumes	windowOpened / windowActivated / windowDeiconified
paint(Graphics) – whenever drawing is needed	paint(Graphics) – same method
stop() – when the page is left	windowDeactivated / windowIconified
destroy() – when the browser unloads it	windowClosing → dispose() → windowClosed

An applet is controlled by its container (browser/appletviewer) through four life-cycle methods; a Frame is an independent window and the application controls its life through window events. (Applets are deprecated for removal in current Java versions; the comparison is for understanding.)

Alternative concept: fetch the doctor list from the database on every department change

Map<String, List<Doctor>> loaded once: time per department change = O(1) hash lookup + O(m_d) to fill the list (m_d = doctors of that department); memory O(d + m) for d departments and m doctors, kept for the whole session; data can become stale. **Query on each change:** one query (round trip, usually milliseconds to tens of milliseconds) + O(m_d) to read rows: time per change is dominated by the network/database; memory O(m_d) only for the current list; always up to date. For a few departments and a rarely changing roster the map is better; if the roster changes often (leave, schedules) a cached map with refresh or a query per change is safer.

TASK 3 – From Sketch to Layout: Campus Library Search Screen (9 marks)

Task 3.1 – Layout managers and justification (3)

Container	Layout	Justification for the resizing requirement
Frame (content pane)	BorderLayout	NORTH and SOUTH keep their preferred height; CENTER receives all remaining space → the results area grows when the window is resized.
North panel (title + search row)	BorderLayout (title NORTH, search row CENTER)	Title keeps its height and stretches in width.
Search row	FlowLayout (left)	Text field, category box and Search button keep their preferred size.
Button row	FlowLayout (centre)	Prev, Next, Reserve keep their size and stay centred.
South panel (buttons + status)	BorderLayout (buttons CENTER, status SOUTH)	Keeps the fixed height needed by the buttons and the status message.

GridLayout was rejected for the whole frame because it would give every region the same size; CardLayout is for alternating screens, which this screen does not have.

Task 3.2 – Nested containers (2)

Nested panel structure with layout managers

```
JFrame  (BorderLayout)
 +-- NORTH : JPanel north (BorderLayout)
 |      +-- NORTH  : JLabel title
 |      +-- CENTER : JPanel searchRow (FlowLayout)  [JLabel][JTextField][JComboBox][JButton Search]
 +-- CENTER : JScrollPane(JTextArea results)        <- grows
 +-- SOUTH  : JPanel south (BorderLayout)
        +-- CENTER : JPanel buttons (FlowLayout)    [Prev][Next][Reserve]
        +-- SOUTH  : JLabel status
```

Task 3.3 – Implementation (3)

Source file: LibrarySearch.java

```
import javax.swing.*;
import java.awt.*;

/** Task 3: Campus library search screen - nested BorderLayout / FlowLayout panels. */
public class LibrarySearch extends JFrame {
    final JTextField query = new JTextField(18);
    final JComboBox<String> category = new JComboBox<>(new String[]{"All", "Title", "Author", "Subject"});
    final JButton search = new JButton("Search"), prev = new JButton("Prev"), next = new JButton("Next"), reserve = new
        JButton("Reserve");
    final JTextArea results = new JTextArea();
    final JLabel status = new JLabel("Enter a search term");
    public LibrarySearch() {
        super("Campus Library");
        setLayout(new BorderLayout(4, 4));                                    // frame: BorderLayout
        JLabel title = new JLabel("Campus Library Search", SwingConstants.CENTER);
        title.setFont(title.getFont().deriveFont(Font.BOLD, 16f));
        JPanel searchRow = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 4)); // search row: FlowLayout (components keep
            their size)
        searchRow.add(new JLabel("Search:")); searchRow.add(query); searchRow.add(category); searchRow.add(search);
        JPanel buttons = new JPanel(new FlowLayout(FlowLayout.CENTER, 8, 4)); // button row: FlowLayout
        buttons.add(prev); buttons.add(next); buttons.add(reserve);
        JPanel south = new JPanel(new BorderLayout());                        // bottom: buttons above the status line
        south.add(buttons, BorderLayout.CENTER); south.add(status, BorderLayout.SOUTH);

        JPanel north = new JPanel(new BorderLayout());                        // search row sits under the title
        north.add(title, BorderLayout.NORTH);        // title bar: NORTH (keeps height, stretches width)
        north.add(searchRow, BorderLayout.CENTER);
        add(north, BorderLayout.NORTH);
        add(new JScrollPane(results), BorderLayout.CENTER);                   // CENTER takes all spare space
        add(south, BorderLayout.SOUTH);

        results.setEditable(false);
        results.setText("1. Introduction to Algorithms\n2. Java: The Complete Reference\n3. Database System Concepts\n4.
            Operating System Concepts\n5. Computer Networks");
        search.addActionListener(e -> status.setText("Searched '" + query.getText() + "' in " + category.getSelectedItem()));
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setSize(520, 330);
    }
    public static void main(String[] args) { SwingUtilities.invokeLater(() -> new LibrarySearch().setVisible(true)); }
}
```

Driver that resizes the window and captures it

```
import javax.swing.*;
import java.awt.*;

public class Task3Shots {
    public static void main(String[] a) throws Exception {
        int[][] sizes = {{420, 260}, {640, 400}, {900, 560}};
        final LibrarySearch[] f = new LibrarySearch[1];
        SwingUtilities.invokeAndWait(() -> { f[0] = new LibrarySearch(); f[0].setLocation(20, 20); f[0].setVisible(true); });
        for (int[] s : sizes) {
            SwingUtilities.invokeAndWait(() -> f[0].setSize(s[0], s[1]));
            Thread.sleep(500);
            Rectangle res = f[0].results.getBounds(), row = f[0].search.getBounds(), btn = f[0].reserve.getBounds();
            System.out.printf("window %dx%d : results area %dx%d | Search button %dx%d | Reserve button %dx%d%n", s[0], s[1],
                 res.width, res.height, row.width, row.height, btn.width, btn.height);
            Shot.screen(new Rectangle(f[0].getX(), f[0].getY(), s[0], s[1]), "shots/task3_" + s[0] + "x" + s[1] + ".png");
        }
        System.exit(0);
    }
}
```

Execution and output (measured component sizes at three window sizes):

```
$ javac LibrarySearch.java Task3Shots.java && java Task3Shots   (virtual display)
window 420x260 : results area 417x148 | Search button 75x25 | Reserve button 82x25
window 640x400 : results area 637x288 | Search button 75x25 | Reserve button 82x25
window 900x560 : results area 897x448 | Search button 75x25 | Reserve button 82x25
```

Task 3.4 – Resizing behaviour (1)

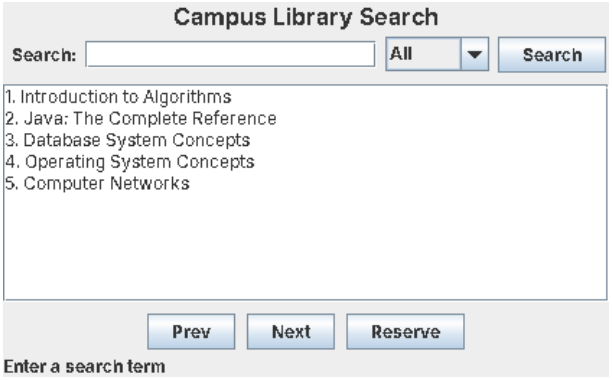


Fig 3.1 – 420 × 260

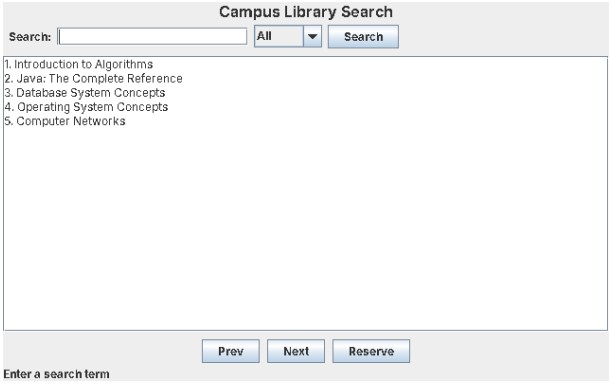


Fig 3.2 – 640 × 400

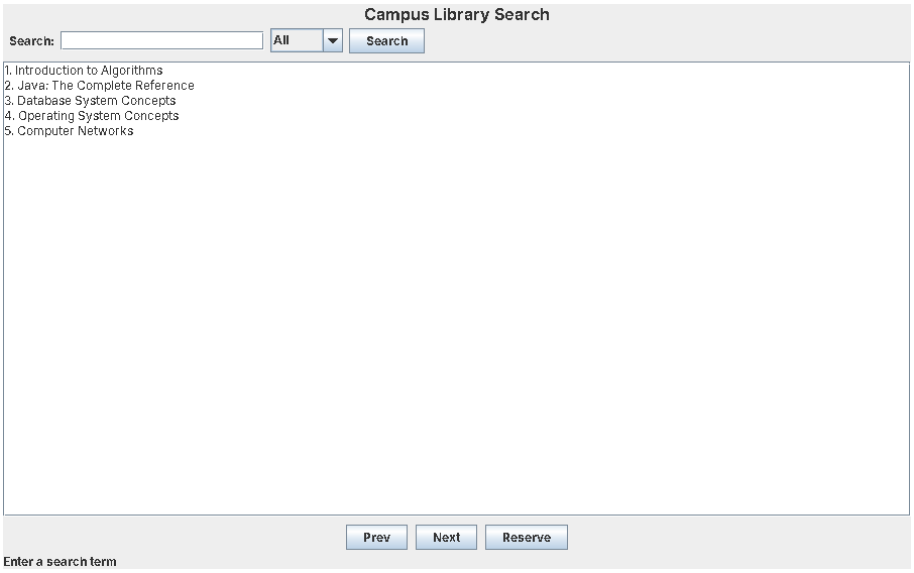


Fig 3.3 – 900 × 560

As designed: the results area grew from 417×148 to 637×288 to 897×448 pixels, while the Search button stayed 75×25 and the Reserve button 82×25 at all three sizes; the title and status line stretched only in width. At 420 px the search row still fits on one line; narrower than that the FlowLayout would wrap onto a second line (and the north panel needs revalidation) – a known limit of FlowLayout.

Alternative concept: GridBagLayout or absolute positioning (setLayout(null) + setBounds)

GridBagLayout places everything in one container with weights (weightx/weighty for the results area), reducing the number of panels (1 container instead of 5) but requiring a GridBagConstraints object per component (more code; each constraint object is extra memory O(c)). **Absolute positioning** uses one container and no layout computation (O(1) per component), but nothing moves or resizes when the window, font, DPI or language changes – the results area would not grow. **Layout computation time** of nested Border/Flow panels is O(c) in the number of components with depth d (each level recomputes preferred sizes → O(c·d)); GridBagLayout is O(c) with a larger constant (it solves column/row weights); null layout has no computation. For 12 components all are imperceptible (< 1 ms). **Space:** nested panels = 5 containers; GridBag = 1 container + c constraints; null = 1 container.

TASK 4 – Event-Handling Forensics: the Drawing Pad (7 marks)

Task 4.1 – Locate the defect (2)

The adapter method is spelt `mouseclicked` (lower-case c); the listener method that AWT calls is `mouseClicked`. Java is case-sensitive, so the code *declares a new, unrelated method* and the inherited empty mouseClicked() of MouseAdapter is the one that is called – the body (count++, drawing) never runs. Because count is never incremented and the label text is never set, the label stays “Clicks: 0” (even if it ran, the label is not updated: the code never calls status.setText).

Task 4.2 – Failing interaction (2)

Source file: FragileDrawPad.java

```
import java.awt.*;
import java.awt.event.*;

/** Task 4.2: first defect fixed (mouseClicked) but dots are drawn once with getGraphics() and not stored. */
public class FragileDrawPad extends Frame {
    int count = 0; Label status = new Label("Clicks: 0");
    public FragileDrawPad() {
        add(status, BorderLayout.SOUTH);
        addMouseListener(new MouseAdapter() {
            @Override public void mouseClicked(MouseEvent e) {
                count++; status.setText("Clicks: " + count);
                Graphics g = getGraphics(); g.fillOval(e.getX(), e.getY(), 6, 6);
            }
        });
        setSize(400, 300);
    }
}
```

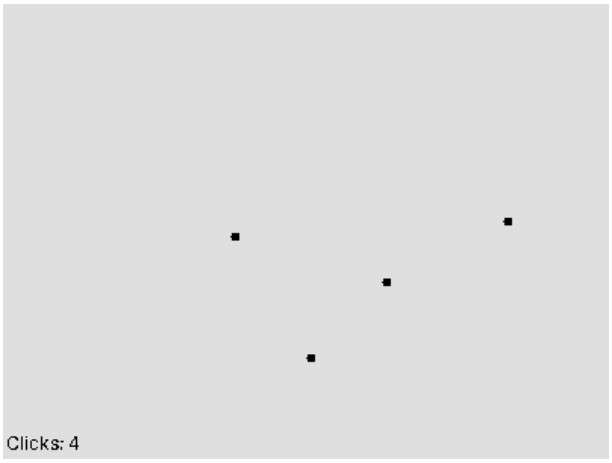


Fig 4.1 – After 4 clicks (defect 1 fixed, dots drawn with getGraphics())



Fig 4.2 – After hiding and re-showing the window: dots gone

Interaction: click four times, then minimise and restore (in the sandbox without a window manager the window was hidden and shown again, which forces the same full repaint). **Why:** getGraphics() draws directly on the current screen surface; nothing is remembered. When the window is restored the system calls paint() (default implementation: clears the background), and since paint() does not know the points, they are not drawn again. Drawing must happen in paint() from stored data.

Task 4.3 – Repair (2) – Complete the Code (TODO 1 and TODO 2)

Source file: DrawPad.java

```
import java.awt.BorderLayout;
import java.awt.Frame;
import java.awt.Graphics;
import java.awt.Label;
import java.awt.Point;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.util.ArrayList;
import java.util.List;
public class DrawPad extends Frame {
    private final List<Point> points = new ArrayList<>();
    private final Label status = new Label("Clicks: 0");

    public DrawPad() {
        super("Draw Pad");
        add(status, BorderLayout.SOUTH);
        addMouseListener(new MouseAdapter() {
            @Override public void mouseClicked(MouseEvent e) {       // TODO 1 completed
                points.add(e.getPoint());
                status.setText("Clicks: " + points.size());
                repaint();
            }
        });
        addWindowListener(new WindowAdapter() {
            @Override public void windowClosing(WindowEvent e) { dispose(); }
        });
        setSize(400, 300);
    }

    @Override public void paint(Graphics g) {
        super.paint(g);
        for (Point p : points) g.fillOval(p.x - 3, p.y - 3, 6, 6);   // TODO 2 completed: 6 x 6 circle centred on the point
    }
    public static void main(String[] args) { new DrawPad().setVisible(true); }
}
```

Driver: real mouse clicks with java.awt.Robot, then screenshot, hide/show, screenshot

```
import java.awt.*;
import java.awt.event.*;

public class DrawPadDemo {
    static void click(Robot r, int x, int y) { r.mouseMove(x, y); r.delay(80); r.mousePress(InputEvent.BUTTON1_DOWN_MASK);
        r.mouseRelease(InputEvent.BUTTON1_DOWN_MASK); r.delay(120); }
    public static void main(String[] a) throws Exception {
        Robot robot = new Robot();
        DrawPad pad = new DrawPad(); pad.setLocation(100, 100); pad.setVisible(true); robot.delay(800);
        Insets in = pad.getInsets();
        int[][] pts = {{150, 150}, {250, 180}, {200, 230}, {330, 140}};
        for (int[] p : pts) click(robot, pad.getX() + p[0], pad.getY() + p[1]);
        robot.delay(300);
        Rectangle r = new Rectangle(pad.getX(), pad.getY(), pad.getWidth(), pad.getHeight());
        Shot.screen(r, "../shots/task4_before.png");
        Label lbl = null; for (Component c : pad.getComponents()) if (c instanceof Label l) lbl = l;
        System.out.println("after 4 clicks, label text = " + lbl.getText());
        // 'minimise and restore' - no window manager in this sandbox, so the window is hidden and shown again (forces a full
             repaint)
        pad.setVisible(false); robot.delay(400); pad.setVisible(true); robot.delay(800);
        Shot.screen(new Rectangle(pad.getX(), pad.getY(), pad.getWidth(), pad.getHeight()), "../shots/task4_after.png");
        System.out.println("after hide/show: label text = " + lbl.getText() + " (dots redrawn from stored points)");
        System.exit(0);
    }
}
```

Execution and output:

```
$ javac DrawPad.java DrawPadDemo.java && java DrawPadDemo   (virtual display)
after 4 clicks, label text = Clicks: 4
after hide/show: label text = Clicks: 4 (dots redrawn from stored points)
```

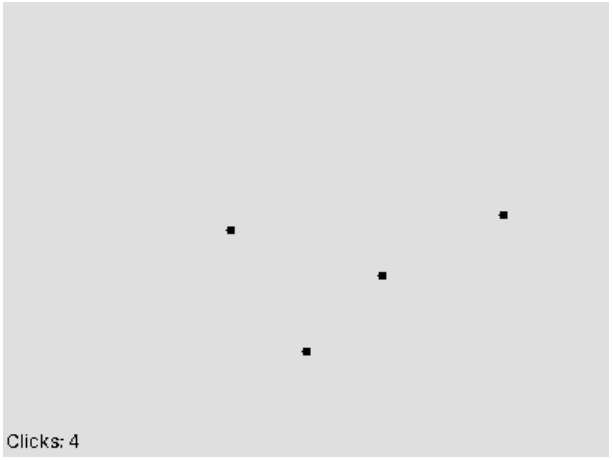


Fig 4.3 – Before minimise: 4 dots, label "Clicks: 4"

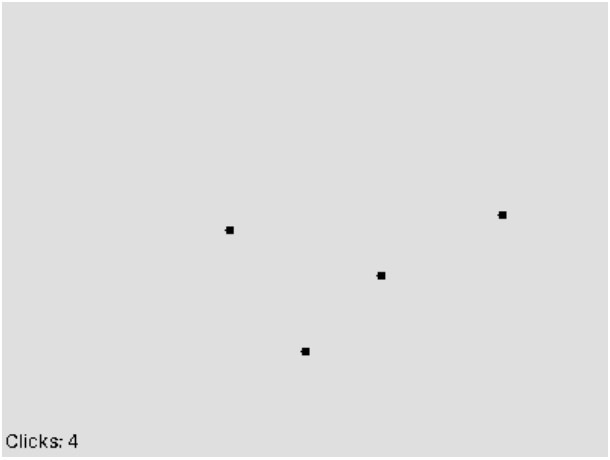


Fig 4.4 – After minimise/restore: dots redrawn from the stored points

Task 4.4 – Prevent recurrence (1)

With `@Override` on the misspelt method the compiler refuses to compile it. Recorded compiler message:

```
$ javac Typo.java   (mouseclicked + @Override)
Typo.java:6: error: method does not override or implement a method from a supertype
           @Override public void mouseclicked(MouseEvent e) { }
           ^
1 error
```

The annotation states the intent "this overrides a supertype method", so a method that overrides nothing is a compile-time error instead of a silent run-time bug.

Alternative concept: draw onto an off-screen BufferedImage and copy it in paint()

Each click draws one dot on the BufferedImage (O(1)) and paint() calls drawImage once. **Time per repaint:** stored points = O(n) (draws n ovals every repaint) versus O(w·h) pixel copy for the image, independent of n (hardware-accelerated, usually fast); for small n (a few hundred dots) the point list is as fast or faster, for thousands of dots the image wins. **Space:** points = O(n) (a Point object of about 16 + 8 bytes each) versus width × height × 4 bytes of pixels, e.g. 400 × 300 × 4 = 480 KB, constant however many clicks. The list keeps the data (can undo, move, resize, save as vector); the image loses it.

TASK 5 – Same Screen, Different Event-Handling Styles: Currency Converter (7 marks)

Task 5.1 – Four implementations (3)

The four versions (all in /home/claude/work/a4/t5): **(a)** the frame implements ActionListener, **(b)** a named inner class, **(c)** an anonymous inner class, **(d)** a lambda; each handles window closing with a WindowAdapter. Only the wiring differs; convert() is identical.

ConverterA.java – (a) frame implements ActionListener

```
public class ConverterA extends JFrame implements ActionListener {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");
    private final JButton convertButton = new JButton("Convert");
    private static double rate(String c) { return c.equals("USD") ? 0.012 : 0.011; }
    public ConverterA() {
        super("Currency Converter");
        setLayout(new FlowLayout());
        add(new JLabel("INR")); add(amount); add(target); add(convertButton); add(result);
        convertButton.addActionListener(this);
        addWindowListener(new WindowAdapter() { @Override public void windowClosing(WindowEvent e) { dispose(); } });
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        pack();
    }
    @Override public void actionPerformed(ActionEvent e) { convert(); }
    void convert() {
        try { double v = Double.parseDouble(amount.getText().trim()) * rate((String) target.getSelectedItem());
              result.setText(String.format("%.2f %s", v, target.getSelectedItem())); }
        catch (NumberFormatException ex) { result.setText("Invalid amount"); }
    }
    public static void main(String[] a) { SwingUtilities.invokeLater(() -> new ConverterA().setVisible(true)); }
}
```

ConverterB.java – (b) named inner class

```
public class ConverterB extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");
    private final JButton convertButton = new JButton("Convert");
    private static double rate(String c) { return c.equals("USD") ? 0.012 : 0.011; }
    public ConverterB() {
        super("Currency Converter");
        setLayout(new FlowLayout());
        add(new JLabel("INR")); add(amount); add(target); add(convertButton); add(result);
        convertButton.addActionListener(new ConvertHandler());
        addWindowListener(new WindowAdapter() { @Override public void windowClosing(WindowEvent e) { dispose(); } });
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        pack();
    }
    private class ConvertHandler implements ActionListener {
        @Override public void actionPerformed(ActionEvent e) { convert(); }
    }
    void convert() {
        try { double v = Double.parseDouble(amount.getText().trim()) * rate((String) target.getSelectedItem());
              result.setText(String.format("%.2f %s", v, target.getSelectedItem())); }
        catch (NumberFormatException ex) { result.setText("Invalid amount"); }
    }

    public static void main(String[] a) { SwingUtilities.invokeLater(() -> new ConverterB().setVisible(true)); }
}
```

ConverterC.java – (c) anonymous inner class

```
public class ConverterC extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");
    private final JButton convertButton = new JButton("Convert");
    private static double rate(String c) { return c.equals("USD") ? 0.012 : 0.011; }
    public ConverterC() {
        super("Currency Converter");
        setLayout(new FlowLayout());
        add(new JLabel("INR")); add(amount); add(target); add(convertButton); add(result);
        convertButton.addActionListener(new ActionListener() { @Override public void actionPerformed(ActionEvent e) {
            convert(); } });

        addWindowListener(new WindowAdapter() { @Override public void windowClosing(WindowEvent e) { dispose(); } });
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        pack();
    }
    void convert() {
        try { double v = Double.parseDouble(amount.getText().trim()) * rate((String) target.getSelectedItem());
              result.setText(String.format("%.2f %s", v, target.getSelectedItem())); }
        catch (NumberFormatException ex) { result.setText("Invalid amount"); }
    }
    public static void main(String[] a) { SwingUtilities.invokeLater(() -> new ConverterC().setVisible(true)); }
}
```

ConverterD.java – (d) lambda

```
public class ConverterD extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");
    private final JButton convertButton = new JButton("Convert");
    private static double rate(String c) { return c.equals("USD") ? 0.012 : 0.011; }
    public ConverterD() {
        super("Currency Converter");
        setLayout(new FlowLayout());
        add(new JLabel("INR")); add(amount); add(target); add(convertButton); add(result);
        convertButton.addActionListener(e -> convert());
        addWindowListener(new WindowAdapter() { @Override public void windowClosing(WindowEvent e) { dispose(); } });
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        pack();
    }
    void convert() {
        try { double v = Double.parseDouble(amount.getText().trim()) * rate((String) target.getSelectedItem());
              result.setText(String.format("%.2f %s", v, target.getSelectedItem())); }
        catch (NumberFormatException ex) { result.setText("Invalid amount"); }
    }

    public static void main(String[] a) { SwingUtilities.invokeLater(() -> new ConverterD().setVisible(true)); }
}
```

Complete the Code – Converter (lambda, TODO 1 and TODO 2)

Source file: Converter.java

```
import java.awt.FlowLayout;
import java.awt.event.ActionListener;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JTextField;
import javax.swing.SwingUtilities;

public class Converter extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target =
            new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");

    private static double rate(String currency) {
        return currency.equals("USD") ? 0.012 : 0.011;
    }
    public Converter() {
        super("Currency Converter");
        setLayout(new FlowLayout());
        JButton convertButton = new JButton("Convert");
        add(new JLabel("INR")); add(amount); add(target);
        add(convertButton); add(result);
        ActionListener l = e -> convert();            // TODO 1 completed: ONE lambda for both triggers
        convertButton.addActionListener(l);
        amount.addActionListener(l);                  // Enter key in the amount field
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        pack();
    }

    void convert() {                                  // TODO 2 completed
        try {
            double v = Double.parseDouble(amount.getText().trim()) * rate((String) target.getSelectedItem());
            result.setText(String.format("%.2f", v) + " " + target.getSelectedItem());
        } catch (NumberFormatException ex) {
            result.setText("Invalid amount");
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Converter().setVisible(true));
    }
}
```

Driver (types a value, presses Enter, captures the window)

```
import javax.swing.*;
import java.awt.*;

public class ConverterShots {
    public static void main(String[] a) throws Exception {
        final Converter[] f = new Converter[1];
        SwingUtilities.invokeAndWait(() -> { f[0] = new Converter(); f[0].setLocation(30, 30); f[0].setSize(420, 70);
            f[0].setVisible(true); });
        JTextField amt = (JTextField) find(f[0].getContentPane(), JTextField.class); JLabel res = null;
        for (Component c : f[0].getContentPane().getComponents()) if (c instanceof JLabel l && l.getText().trim().isEmpty())
            res = l;
        final JLabel result = res;
        SwingUtilities.invokeAndWait(() -> { amt.setText("5000"); amt.postActionEvent(); });
        Thread.sleep(400); System.out.println("valid  : 5000 INR (Enter key) -> " + result.getText());
        Shot.screen(new Rectangle(f[0].getX(), f[0].getY(), 420, 70), "../shots/task5_valid.png");
        SwingUtilities.invokeAndWait(() -> { amt.setText("12x"); amt.postActionEvent(); });
        Thread.sleep(400); System.out.println("invalid: 12x           -> " + result.getText());
        Shot.screen(new Rectangle(f[0].getX(), f[0].getY(), 420, 70), "../shots/task5_invalid.png");
        System.exit(0);
    }
    static Component find(Container c, Class<?> k) { for (Component x : c.getComponents()) if (k.isInstance(x)) return x;
        return null; }
}
```

Execution and output:

```
$ javac Converter.java ConverterShots.java && java ConverterShots   (virtual display)
valid  : 5000 INR (Enter key) -> 60.00 USD
invalid: 12x           -> Invalid amount
```





Fig 5.1 – Valid conversion: 5000 INR → 60.00 USD (Enter key)

Fig 5.2 – Invalid amount “12x”

Task 5.2 – Change the requirement (2)

New requirement: convert also on Enter in the amount field, and show the result in a dialog box. The after-versions (ConverterA2 … D2) were produced and compared line by line with the originals (class renames ignored):

Style	Lines added	Lines removed	Total changed
(a)	4	2	6
(b)	4	2	6
(c)	4	2	6
(d)	6	3	9

Styles (a), (b), (c) each needed 1 new registration line for the Enter key and the dialog change inside convert() (3 lines modified). Style (d) was restructured to share one named lambda `ActionListener l = e -> convert();` for both sources, which counts as more changed lines but gives one listener object for both triggers; a one-line lambda per source would be 1 added line as well. In all four versions the change is small because convert() is a single shared method – the key design decision. Where the handler logic is inside the listener (the original broken FeeWindow) the same change would be much larger.

Task 5.3 – Compare the styles (2)

Style	Readability	Access to frame fields	Reuse of the handler	Ease of change
(a) frame implements	One actionPerformed for all sources → needs if/else on e.getSource()	Direct	Low (one method for everything)	Grows with every source
(b) named inner class	Clear name, separate class	Direct (inner class sees outer fields)	High (new Handler() for several sources)	Good
(c) anonymous class	Verbose but local to where it is used	Direct	Low (cannot be reused)	Good for one-off handlers
(d) lambda	Shortest, reads like the action	Direct	High if stored in a variable or method reference	Excellent

For a large application I would use **lambdas / method references that call methods of a controller** (or Swing Actions for items that appear as button + menu + shortcut): handlers are short, the real logic lives in named, testable methods, and each source can have its own handler; style (a) does not scale because it forces a dispatch chain, and anonymous classes are only needed when the listener interface has several methods (then use an adapter).

Alternative concept: method reference (this::convert) or one shared Action object

`convertButton.addActionListener(e -> convert())` becomes `... (this::convert)`-style (with an ActionEvent parameter) – the same cost as a lambda: one small object per registration (a non-capturing lambda is created once and shared). An **Action** object (AbstractAction) is created **once** and used for the button, a menu item and a key binding: 1 listener object for k triggers (space O(1) instead of O(k)); adding a new trigger is one line (*item.setAction(convertAction)*) and automatically inherits text, enabled state and shortcut. The four implemented styles create one listener object per source (O(k)) and need one registration line per trigger.

TASK 6 – Design Challenge: Campus Event Registration Desktop Application (13 marks)

Stage 1 – Interpret the problem (2)

- **Users:** a student (registers through the form) and a committee member (views, filters, exports).
- **Screens:** main window with registration form + table + filter + paging; dialogs for messages, About and the file chooser for export.
- **Data:** registration(id, reg_no, name, email, event_name, created_at) in MySQL.
- **Testable requirements:** R1 a valid form creates exactly one row; R2 empty/invalid reg no, name, e-mail or event is rejected with a clear message and no row; R3 the same (reg_no, event) cannot be stored twice – enforced by the service check *and* by a UNIQUE constraint; R4 the same student may register for different events; R5 the window stays responsive while loading (database work off the EDT); R6 the table filters by event and shows pages of 10 rows; R7 Export writes a CSV of the filtered rows.

Stage 2 – Screen and event flow (2)

Main screen sketch

```
+-------------------------------------------------------------------------------+
| File  Help                                                      (JMenuBar)  |
+-------------------------------------------------------------------------------+
| Reg No [_________] Name [__________] E-mail [_______________] Event [v] [Register]|
| Filter by event: [All events v]   [Prev] Page 1 of N (rows)  [Next]              |
+-------------------------------------------------------------------------------+
| ID | Reg No | Name | E-mail | Event                    (JTable, JScrollPane)   |
+-------------------------------------------------------------------------------+
| status line                                                                   |
+-------------------------------------------------------------------------------+
```

Flow between screens and dialogs

```
Start -> Main window (loads page 1 in a SwingWorker)
Main window --Register--> validation fails -> status shows the error (stay)
                     \-> duplicate -> status shows "already registered" (stay)
                     \-> success -> clear form, reload table
Main window --Filter / Prev / Next--> reload page (SwingWorker)
Main window --File > Export CSV--> file chooser dialog -> write file in SwingWorker -> status
Main window --Help > About--> message dialog
Main window --File > Exit / close box--> dispose / exit
```

Stage 3 – Critical design decisions (2)

Concern	Where it is done	How
Input validation	RegistrationService.register()	regex for 9-digit reg no and e-mail, name length, event in list – independent of the UI so it can be tested; the UI shows the message
Duplicate checking	Service (friendly message) + database UNIQUE(reg_no, event_name)	The check-then-insert has a race if two desks register at once; the constraint is the final guard and its error is converted to the same friendly message
Database access	RegistrationDao only	PreparedStatement everywhere; try-with-resources; no SQL in the UI or the service
Long-running work off the EDT	SwingWorker in EventApp	doInBackground() calls the service; done() updates the table on the EDT; buttons are disabled while a job runs; paging (LIMIT/OFFSET) limits rows per query

Stage 4 – Classes and handlers (4)

Class diagram: three layers

```
   UI layer                    Service layer                  Data-access (JDBC) layer
+-----------------+        +-----------------------+        +-------------------------+
| EventApp        | -----> | RegistrationService   | -----> | RegistrationDao         |
| (JFrame, JTable,|        | register(...) : Result |        | exists(reg,event)       |
|  JMenuBar,      | <----- | page(event,page,size)  | <----- | insert(Registration)    |
|  SwingWorker)   | Result | count(event)           | rows   | page(event,limit,offset) |
+-----------------+        +-----------------------+        | count(event)            |
                                                            +-----------+-------------+
                         Registration (record)                          | JDBC
                                                                  MySQL: registration table
```

Pseudocode – Register action

```
ON ActionEvent FROM registerButton
BEGIN
  read regNo, name, email, event from the form
  registerButton.disable(); status <- "Registering..."
  RUN IN BACKGROUND:   result <- service.register(regNo, name, email, event)
     service.register:
        IF regNo invalid OR name invalid OR email invalid OR event invalid THEN RETURN failure(message)
        IF dao.exists(regNo, event) THEN RETURN failure("already registered")
        TRY dao.insert(...) RETURN success
        CATCH unique-constraint violation -> RETURN failure("already registered")   // race with another desk
        CATCH other SQLException -> RETURN failure("Database error")
  WHEN DONE (event thread):
     status <- result.message
     IF result.ok THEN clear the form; reload the current page
     registerButton.enable()
END
```

Pseudocode – Filter action

```
ON ItemEvent/ActionEvent FROM filterCombo
BEGIN
  pageNo <- 0; event <- selected filter (null for "All events")
  RUN IN BACKGROUND:  rows <- service.page(event, pageNo, 10)    // SELECT ... WHERE event_name=? ORDER BY id LIMIT 10
      OFFSET pageNo*10
                      total <- service.count(event)
  WHEN DONE:  table.model <- rows; pageLabel <- "Page p of N (total rows)"; enable Prev/Next accordingly
END
```

Stage 5 – Implement and demonstrate (3)

SQL script for MySQL (the application connects with *jdbc:mysql://localhost:3306/campus_events*, user and password as arguments); the tests below ran the same classes against SQLite because no MySQL server was available in the sandbox.

Source file: schema_mysql.sql

```
-- Task 6: Campus Event Registration (MySQL 8). Run once:  mysql -u root -p < schema_mysql.sql
CREATE DATABASE IF NOT EXISTS campus_events;
USE campus_events;

CREATE TABLE IF NOT EXISTS registration (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    reg_no     VARCHAR(9)   NOT NULL,
    name       VARCHAR(60)  NOT NULL,
    email      VARCHAR(80)  NOT NULL,
    event_name VARCHAR(40)  NOT NULL,
    created_at TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_student_event UNIQUE (reg_no, event_name)   -- a student cannot register twice for the same event
);
CREATE INDEX idx_registration_event ON registration(event_name);   -- speeds up the filter-by-event query
```

Source file: Registration.java

```
/** Model: one row of the registration table. */
public record Registration(int id, String regNo, String name, String email, String eventName) {}
```

Source file: RegistrationDao.java

```
import java.sql.*;
import java.util.*;

/** Data-access layer: the ONLY class that contains SQL / JDBC. */
public class RegistrationDao {
    private final String url, user, pwd;
    public RegistrationDao(String url, String user, String pwd) { this.url = url; this.user = user; this.pwd = pwd; }
    private Connection open() throws SQLException { return user == null ? DriverManager.getConnection(url) :
        DriverManager.getConnection(url, user, pwd); }

    public boolean exists(String regNo, String event) throws SQLException {
        try (Connection c = open(); PreparedStatement ps = c.prepareStatement("SELECT 1 FROM registration WHERE reg_no=? AND
            event_name=?")) {
            ps.setString(1, regNo); ps.setString(2, event);
            try (ResultSet rs = ps.executeQuery()) { return rs.next(); }
        }
    }
    public void insert(Registration r) throws SQLException {
        try (Connection c = open(); PreparedStatement ps = c.prepareStatement("INSERT INTO
            registration(reg_no,name,email,event_name) VALUES (?,?,?,?)")) {
            ps.setString(1, r.regNo()); ps.setString(2, r.name()); ps.setString(3, r.email()); ps.setString(4,
                r.eventName());
            ps.executeUpdate();
        }
    }
    /** One page of rows; event == null means all events. */
    public List<Registration> page(String event, int limit, int offset) throws SQLException {
        String sql = "SELECT id,reg_no,name,email,event_name FROM registration" + (event == null ? "" : " WHERE
            event_name=?") + " ORDER BY id LIMIT ? OFFSET ?";
        List<Registration> out = new ArrayList<>();
        try (Connection c = open(); PreparedStatement ps = c.prepareStatement(sql)) {
            int i = 1; if (event != null) ps.setString(i++, event);
            ps.setInt(i++, limit); ps.setInt(i, offset);
            try (ResultSet rs = ps.executeQuery()) { while (rs.next()) out.add(new Registration(rs.getInt(1),
                rs.getString(2), rs.getString(3), rs.getString(4), rs.getString(5))); }
        }
        return out;
    }
    public int count(String event) throws SQLException {
        try (Connection c = open(); PreparedStatement ps = c.prepareStatement("SELECT COUNT(*) FROM registration" + (event ==
             null ? "" : " WHERE event_name=?"))) {
            if (event != null) ps.setString(1, event);
            try (ResultSet rs = ps.executeQuery()) { rs.next(); return rs.getInt(1); }
        }
    }
}
```

Source file: RegistrationService.java

```
import java.sql.SQLException;
import java.util.*;

/** Service layer: validation + duplicate rule. No Swing, no SQL. */
public class RegistrationService {
    public record Result(boolean ok, String message) {}
    public static final List<String> EVENTS = List.of("Dance", "Hackathon", "Quiz", "Music");
    private final RegistrationDao dao;
    public RegistrationService(RegistrationDao dao) { this.dao = dao; }

    public Result register(String regNo, String name, String email, String event) {
        if (regNo == null || !regNo.trim().matches("\\d{9}")) return new Result(false, "Register number must be 9 digits");
        if (name == null || name.trim().length() < 2) return new Result(false, "Enter the student's name");
        if (email == null || !email.trim().matches("[\\w.+-]+@[\\w-]+(\\.[\\w-]+)+")) return new Result(false, "Enter a valid
             e-mail address");
        if (event == null || !EVENTS.contains(event)) return new Result(false, "Select an event");
        try {
            if (dao.exists(regNo.trim(), event)) return new Result(false, regNo.trim() + " is already registered for " +
                event);   // friendly duplicate check
            dao.insert(new Registration(0, regNo.trim(), name.trim(), email.trim(), event));
            return new Result(true, "Registered " + regNo.trim() + " for " + event);
        } catch (SQLException e) {
            // the UNIQUE (reg_no, event_name) constraint is the final guard when two desks insert at the same moment
            return new Result(false, e.getMessage() != null && e.getMessage().toLowerCase().contains("unique") ? regNo.trim()
                 + " is already registered for " + event : "Database error: " + e.getMessage());
        }
    }
    public List<Registration> page(String event, int pageNo, int size) throws SQLException { return dao.page(event, size,
        pageNo * size); }
    public int count(String event) throws SQLException { return dao.count(event); }
}
```

Source file: EventApp.java

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.io.*;
import java.nio.file.*;
import java.sql.SQLException;
import java.util.List;

/** View layer: Swing only. Database work runs in SwingWorker threads, never on the event-dispatch thread. */
public class EventApp extends JFrame {
    static final int PAGE = 10;
    final RegistrationService service;
    final JTextField regNo = new JTextField(10), name = new JTextField(12), email = new JTextField(16);
    final JComboBox<String> event = new JComboBox<>(RegistrationService.EVENTS.toArray(new String[0]));
    final JComboBox<String> filter = new JComboBox<>();
    final DefaultTableModel model = new DefaultTableModel(new String[]{"ID", "Reg No", "Name", "E-mail", "Event"}, 0) {
        public boolean isCellEditable(int r, int c) { return false; } };
    final JTable table = new JTable(model);
    final JLabel status = new JLabel("Ready"), pageInfo = new JLabel("Page 1");
    final JButton register = new JButton("Register"), prev = new JButton("Prev"), next = new JButton("Next");
    int pageNo = 0, total = 0;

    public EventApp(RegistrationService service) {
        super("Campus Event Registration");
        this.service = service;
        JMenuBar bar = new JMenuBar(); JMenu file = new JMenu("File"), help = new JMenu("Help");
        JMenuItem export = new JMenuItem("Export CSV..."), exit = new JMenuItem("Exit"), about = new JMenuItem("About");
        file.add(export); file.addSeparator(); file.add(exit); help.add(about); bar.add(file); bar.add(help);
            setJMenuBar(bar);
        JPanel form = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 4));
        form.add(new JLabel("Reg No")); form.add(regNo); form.add(new JLabel("Name")); form.add(name);
        form.add(new JLabel("E-mail")); form.add(email); form.add(new JLabel("Event")); form.add(event); form.add(register);
        JPanel filterRow = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 4));
        filter.addItem("All events"); for (String e : RegistrationService.EVENTS) filter.addItem(e);
        filterRow.add(new JLabel("Filter by event:")); filterRow.add(filter); filterRow.add(prev); filterRow.add(pageInfo);
            filterRow.add(next);
        JPanel top = new JPanel(new BorderLayout()); top.add(form, BorderLayout.NORTH); top.add(filterRow,
            BorderLayout.SOUTH);
        add(top, BorderLayout.NORTH); add(new JScrollPane(table), BorderLayout.CENTER); add(status, BorderLayout.SOUTH);

        register.addActionListener(e -> doRegister());
        filter.addActionListener(e -> { pageNo = 0; load(); });
        prev.addActionListener(e -> { if (pageNo > 0) { pageNo--; load(); } });
        next.addActionListener(e -> { if ((pageNo + 1) * PAGE < total) { pageNo++; load(); } });
        export.addActionListener(e -> doExport());
        exit.addActionListener(e -> dispose());
        about.addActionListener(e -> JOptionPane.showMessageDialog(this, "Campus Event Registration v1.0"));
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setSize(860, 380);
    }
    String selectedFilter() { String s = (String) filter.getSelectedItem(); return s == null || s.startsWith("All") ? null :
        s; }
    void doRegister() {
        String r = regNo.getText(), n = name.getText(), m = email.getText(), ev = (String) event.getSelectedItem();
        register.setEnabled(false); status.setText("Registering...");
        new SwingWorker<RegistrationService.Result, Void>() {
            protected RegistrationService.Result doInBackground() { return service.register(r, n, m, ev); }
            protected void done() {
                try { RegistrationService.Result res = get(); status.setText((res.ok() ? "OK: " : "Error: ") +
                    res.message()); if (res.ok()) { regNo.setText(""); name.setText(""); email.setText(""); load(); } }
                catch (Exception ex) { status.setText("Unexpected error"); }
                register.setEnabled(true);
            }
        }.execute();
    }
    void load() {
        String f = selectedFilter(); status.setText("Loading...");
        new SwingWorker<Object[], Void>() {
            protected Object[] doInBackground() throws SQLException { return new Object[]{service.page(f, pageNo, PAGE),
                service.count(f)}; }
            @SuppressWarnings("unchecked") protected void done() {
                try {
                    Object[] r = get(); List<Registration> rows = (List<Registration>) r[0]; total = (Integer) r[1];
                    model.setRowCount(0);
                    for (Registration x : rows) model.addRow(new Object[]{x.id(), x.regNo(), x.name(), x.email(),
                        x.eventName()});
                    int pages = Math.max(1, (total + PAGE - 1) / PAGE);
                    pageInfo.setText("Page " + (pageNo + 1) + " of " + pages + "  (" + total + " rows)");
                    prev.setEnabled(pageNo > 0); next.setEnabled((pageNo + 1) * PAGE < total); status.setText("Loaded " +
                        rows.size() + " rows");
                } catch (Exception ex) { status.setText("Could not load: " + ex.getMessage()); }
            }
        }.execute();
    }
    void doExport() {
        JFileChooser fc = new JFileChooser(); fc.setSelectedFile(new File("registrations.csv"));
        if (fc.showSaveDialog(this) != JFileChooser.APPROVE_OPTION) return;
        Path p = fc.getSelectedFile().toPath(); String f = selectedFilter(); status.setText("Exporting...");
```

```
        new SwingWorker<Integer, Void>() {
            protected Integer doInBackground() throws Exception {
                int n = 0, off = 0;
                try (BufferedWriter w = Files.newBufferedWriter(p)) {
                    w.write("id,reg_no,name,email,event\n");
                    List<Registration> rows;
                    while (!(rows = service.page(f, off / 500, 500)).isEmpty()) { for (Registration x : rows) {
                        w.write(x.id() + "," + x.regNo() + "," + x.name() + "," + x.email() + "," + x.eventName() + "\n");
                        n++; } off += 500; }
                }
                return n;
            }
            protected void done() { try { status.setText("Exported " + get() + " rows to " + p); } catch (Exception ex) {
                status.setText("Export failed: " + ex.getMessage()); } }
        }.execute();
    }
    public static void main(String[] args) {
        String url = args.length > 0 ? args[0] : "jdbc:mysql://localhost:3306/campus_events";
        String user = args.length > 1 ? args[1] : "root", pwd = args.length > 2 ? args[2] : "";
        SwingUtilities.invokeLater(() -> { EventApp a = new EventApp(new RegistrationService(new RegistrationDao(url, user,
            pwd))); a.setVisible(true); a.load(); });
    }
}
```

Test driver: test cases, paging measurement, window screenshots

```
import javax.swing.*;
import java.awt.*;
import java.sql.*;
import java.util.*;
import java.util.List;
public class Task6Demo {
    static final String DDL = "CREATE TABLE registration (id INTEGER PRIMARY KEY AUTOINCREMENT, reg_no TEXT NOT NULL, name
        TEXT NOT NULL, email TEXT NOT NULL, event_name TEXT NOT NULL, created_at TEXT DEFAULT CURRENT_TIMESTAMP, UNIQUE
        (reg_no, event_name))";
    public static void main(String[] a) throws Exception {
        String url = "jdbc:sqlite:" + java.nio.file.Files.createTempFile("events", ".db");
        try (Connection c = DriverManager.getConnection(url); Statement st = c.createStatement()) { st.execute(DDL);
            st.execute("CREATE INDEX idx_event ON registration(event_name)"); }
        RegistrationService svc = new RegistrationService(new RegistrationDao(url, null, null));
        System.out.println("== Stage 5 test cases ==");
        String[][] cases = {
            {"NORMAL        ", "192311001", "Arun Kumar", "arun@college.edu", "Hackathon"},
            {"NORMAL (2nd event)", "192311001", "Arun Kumar", "arun@college.edu", "Quiz"},
            {"NORMAL (other student)", "192311002", "Divya S", "divya@college.edu", "Hackathon"},
            {"DUPLICATE     ", "192311001", "Arun Kumar", "arun@college.edu", "Hackathon"},
            {"INVALID reg no", "1923", "Ravi", "ravi@college.edu", "Dance"},
            {"INVALID e-mail", "192311003", "Ravi", "ravi-at-college", "Dance"},
            {"INVALID empty name", "192311004", " ", "x@college.edu", "Dance"},
            {"INVALID event ", "192311005", "Meena", "meena@college.edu", "Chess"},
        };
        for (String[] c : cases) System.out.printf("  %-24s %-10s %-10s -> %s%n", c[0], c[1], c[4], svc.register(c[1], c[2],
            c[3], c[4]));
        System.out.println("  rows stored: " + svc.count(null) + " (expected 3)");
        // DB-level guard: bypass the service check and insert the duplicate directly
        try { new RegistrationDao(url, null, null).insert(new Registration(0, "192311001", "Arun Kumar", "arun@college.edu",
            "Hackathon")); System.out.println("  direct duplicate insert: ACCEPTED (BUG)"); }
        catch (SQLException e) { System.out.println("  direct duplicate insert rejected by UNIQUE constraint: " +
            e.getMessage().split("\n")[0]); }

        // seed for the demo window and the paging analysis
        try (Connection c = DriverManager.getConnection(url)) {
            c.setAutoCommit(false);
            try (PreparedStatement ps = c.prepareStatement("INSERT INTO registration(reg_no,name,email,event_name) VALUES
                (?,?,?,?)")) {
                Random r = new Random(1);
                for (int i = 0; i < 100_000; i++) { ps.setString(1, String.valueOf(192_400_000 + i)); ps.setString(2,
                    "Student " + i); ps.setString(3, "s" + i + "@college.edu"); ps.setString(4,
                    RegistrationService.EVENTS.get(r.nextInt(4))); ps.addBatch(); if (i % 5000 == 0) ps.executeBatch(); }
                ps.executeBatch();
            }
            c.commit();
        }
        System.out.println("\n== Paging vs loading everything (100,003 rows) ==");
        RegistrationDao dao = new RegistrationDao(url, null, null);
        for (int rep = 0; rep < 2; rep++) { dao.page(null, 10, 0); }
        long t = System.nanoTime(); List<Registration> pg = dao.page(null, 10, 50_000); double pageMs = (System.nanoTime() -
            t) / 1e6;
        t = System.nanoTime(); List<Registration> pg0 = dao.page(null, 10, 0); double page0Ms = (System.nanoTime() - t) /
            1e6;
        System.gc(); long m0 = used();
        t = System.nanoTime(); List<Registration> all = dao.page(null, Integer.MAX_VALUE, 0); double allMs =
            (System.nanoTime() - t) / 1e6;
        long m1 = used();
        System.out.printf("  one page of 10 rows at OFFSET 0     : %.2f ms%n  one page of 10 rows at OFFSET 50,000 : %.2f
            ms%n  ALL %d rows into memory          : %.0f ms, about %d MB of heap%n", page0Ms, pageMs, all.size(), allMs,
            Math.max(0, (m1 - m0) / (1024 * 1024)));
        // keep the UI thread free: measure how long a blocking load would freeze the window
        System.out.printf("  a blocking load of all rows would freeze the window for ~%.1f s; a SwingWorker page load keeps
            it responsive%n", allMs / 1000);
        all = null;
        final EventApp[] app = new EventApp[1];
        SwingUtilities.invokeAndWait(() -> { app[0] = new EventApp(svc); app[0].setLocation(10, 10); app[0].setVisible(true);
             app[0].load(); });
        Thread.sleep(1500);
        Shot.screen(new Rectangle(10, 10, 860, 380), "../shots/task6_main.png");
        System.out.println("\nwindow status line: " + app[0].status.getText() + " | " + app[0].pageInfo.getText());
        SwingUtilities.invokeAndWait(() -> { app[0].regNo.setText("192311001"); app[0].name.setText("Arun Kumar");
            app[0].email.setText("arun@college.edu"); app[0].event.setSelectedItem("Hackathon"); app[0].register.doClick();
            });
        Thread.sleep(800); Shot.screen(new Rectangle(10, 10, 860, 380), "../shots/task6_duplicate.png");
        System.out.println("duplicate attempt in the window -> " + app[0].status.getText());
        SwingUtilities.invokeAndWait(() -> { app[0].filter.setSelectedItem("Quiz"); });
        Thread.sleep(800); Shot.screen(new Rectangle(10, 10, 860, 380), "../shots/task6_filter.png");
        System.out.println("filter Quiz -> " + app[0].pageInfo.getText());
        System.exit(0);
    }
    static long used() { Runtime r = Runtime.getRuntime(); return r.totalMemory() - r.freeMemory(); }
}
```

Execution and output:

```
$ javac -cp sqlite-jdbc.jar *.java && java -cp .:sqlite-jdbc.jar Task6Demo   (virtual display)
== Stage 5 test cases ==
  NORMAL                192311001  Hackathon  -> Result[ok=true, message=Registered 192311001 for Hackathon]
  NORMAL (2nd event)    192311001  Quiz       -> Result[ok=true, message=Registered 192311001 for Quiz]
  NORMAL (other student)   192311002  Hackathon  -> Result[ok=true, message=Registered 192311002 for Hackathon]
  DUPLICATE             192311001  Hackathon  -> Result[ok=false, message=192311001 is already registered for Hackathon]
```

```
  INVALID reg no          1923       Dance      -> Result[ok=false, message=Register number must be 9 digits]
  INVALID e-mail          192311003  Dance      -> Result[ok=false, message=Enter a valid e-mail address]
  INVALID empty name      192311004  Dance      -> Result[ok=false, message=Enter the student's name]
  INVALID event           192311005  Chess      -> Result[ok=false, message=Select an event]
  rows stored: 3 (expected 3)
  direct duplicate insert rejected by UNIQUE constraint: [SQLITE_CONSTRAINT_UNIQUE]  A UNIQUE constraint failed (UNIQUE
      constraint failed: registration.reg_no, registration.event_name)

== Paging vs loading everything (100,003 rows) ==
  one page of 10 rows at OFFSET 0     : 4.39 ms
  one page of 10 rows at OFFSET 50,000 : 4.13 ms
  ALL 100003 rows into memory           : 353 ms, about 32 MB of heap
  a blocking load of all rows would freeze the window for ~0.4 s; a SwingWorker page load keeps it responsive

window status line: Loaded 10 rows | Page 1 of 10001  (100003 rows)
duplicate attempt in the window -> Error: 192311001 is already registered for Hackathon
filter Quiz -> Page 1 of 2477  (24769 rows)
```

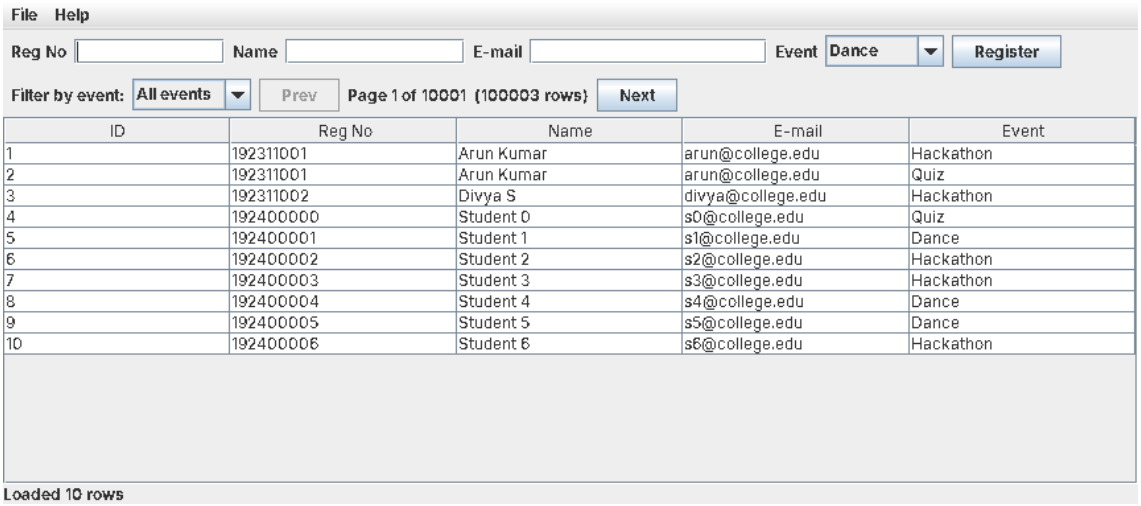


Fig 6.1 – Main window with the first page of 100,003 registrations loaded by a SwingWorker

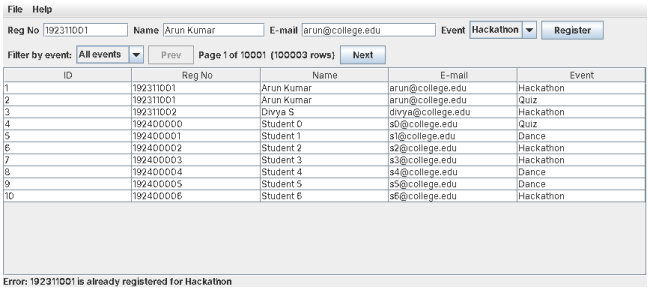


Fig 6.2 – Duplicate registration rejected: "192311001 is already registered for Hackathon"

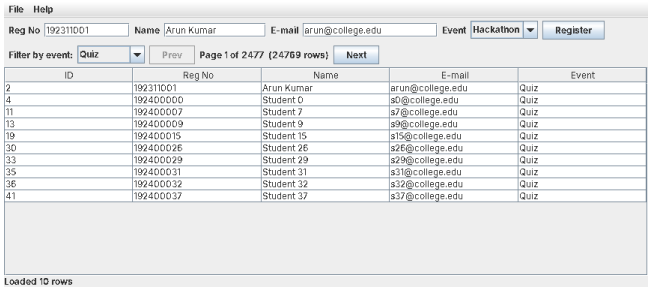


Fig 6.3 – Filtered by event "Quiz": 24,769 rows, page 1 of 2,477

Test	Input	Expected	Observed
Normal	192311001 / Hackathon	registered	registered
Normal, other event	192311001 / Quiz	registered	registered
Duplicate	192311001 / Hackathon again	rejected	rejected (and DB constraint also rejects a direct insert)
Invalid	reg no "1923"; e-mail without @; blank name; unknown event	rejected with a message	4 distinct messages
Rows stored	after the tests	3	3

Alternative concept: load all registrations into the JTable at once, instead of paging with LIMIT/OFFSET in a SwingWorker

Loading all 100,003 rows: measured 353 ms and about 32 MB of heap for the list (more for the JTable model: one Object[] per row); if done on the EDT the window would be frozen for that time (and for much longer over a network), and 1,000,000 rows would need about 320 MB. **Paging:** one page of 10 rows took about 4 ms at OFFSET 0 and 4 ms at OFFSET 50,000 here (SQLite; in MySQL a large OFFSET still scans and discards the skipped rows – O(offset + limit) – so very deep pages get slower; keyset pagination (WHERE id > last) avoids that); memory O(page size) = a few KB; the window stays responsive because the worker runs off the EDT. Time: all-at-once O(n) per load; paging O(offset + limit) per page, but only O(limit) rows travel to the client. The price of paging is more queries and a count query; the filter uses the index on event_name.

SUMMARY OF PROGRAMS EXECUTED

Task	Programs	Result
1	FeeService, FeeWindow, Task1Demo	Validation, payment and window tested; UI never blocks
2	Task24Trace	Event order recorded (9 steps)
3	LibrarySearch, Task3Shots	Results area grows, buttons keep size
4	BrokenDrawPad, FragileDrawPad, DrawPad	Defects reproduced and fixed; compiler message recorded
5	ConverterA–D, A2–D2, Converter	Four styles + change effort; lambda version complete
6	EventApp + service + DAO + SQL	Normal, invalid, duplicate cases; paging measured

Note: add your own screenshots of the programs on your computer (compiler, window, output) next to the figures; if MySQL is available, run schema_mysql.sql and start EventApp with the JDBC URL, user and password.